AI for All - Tools

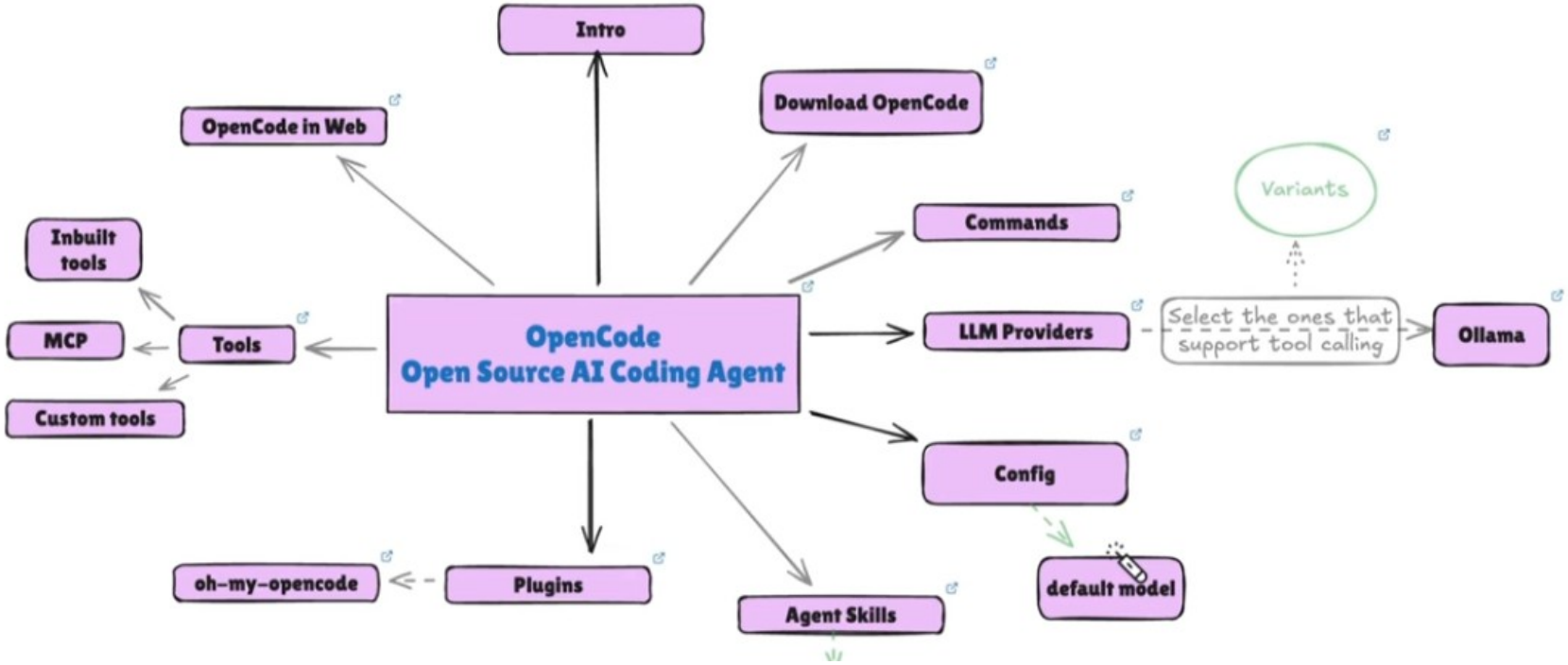
Yogesh Haribhau Kulkarni

YHK

OpenCode

YHK

OpenCode Ecosystem



(Ref: OpenCode CRASH Course - Data Science Basics)

What is OpenCode?

- Open-source coding agent built for the terminal (TUI).
- **IDE Integration possible**: Works with VS Code, Cursor, and any terminal-supporting IDE
- **GitHub Integration**: Automate issues and PRs directly from comments
- **Model-Agnostic:** Supports 75+ AI models (OpenAI, Claude, Groq, Ollama, and more).
  - *Local:* Ollama (fully offline, zero data egress)
  - *Cloud:* OpenAI, Claude, Groq, Gemini, etc.
- **Privacy-Focused:** Runs locally or uses your own API keys, your code stays on your machine.
- **Built With:** Rust/Go for performance; Bubble Tea for the TUI layer.

YHK

Capabilities

Think of it as having a senior developer sitting next to you

- Writing and debugging code
- Understanding existing codebases
- Refactoring and optimization
- Answering technical questions
- Automating repetitive tasks

YHK

Key Features

- **Interactive TUI:** Smooth terminal UI built with Bubble Tea; keyboard-driven workflow.
- **Contextual Understanding:** Reads your entire repo, understands dependencies, structure, languages.
- **Agentic Workflows:** Plans changes, searches files, executes shell commands, modifies code autonomously.
- **Plan / Build Modes/Agents:** Review proposed changes before applying, deterministic control.
- **Slash Commands:** `/init`, `/help`, `/models`, `/share`, and more.
- **Context Mentions:** `@file:`, `@workspace` for scoped AI queries.

YHK

Core Integration

- **LSP Integration:** Language Server Protocol for real-time code intelligence (completions, diagnostics, go-to-def).
- **MCP (Model Context Protocol):** Plug in external tools and data sources to extend agent capabilities.
- **ACP Support:** Agent Communication Protocol for inter-agent coordination.
- **Agent Skills & Custom Tools:** Define reusable agent behaviors and wire in custom shell/API tools.
- **Rules & Permissions:** Fine-grained control over what the agent can read, write, or execute.
- **Multi-Provider Routing:** Failover and cost-based routing across model providers.

YHK

OpenCode vs Other Tools

Feature	OpenCode	Claude Code	Cursor
Open Source	✅	❌	❌
Free Tier	✅	Limited	Limited
Terminal-first	✅	✅	❌
Provider Agnostic	✅	❌ (Claude only)	❌
Local Models	✅	❌	✅
IDE Integration	✅	✅	Native

(Ref: OpenCode - nxcode.io

Installation

- **Quick Install (Recommended):**

  ```
  $ curl -fsSL https://opencode.ai/install | sh
  ```

- **Install npm/bun:** `npm i -g opencode-ai@latest` or `bun add -g opencode-ai`
- **Desktop App:** Download directly from opencode.ai/download for a native desktop experience.

YHK

Getting Started

- **First Step:** Run `/init` to ground the agent in your project.
- **Configure Provider**: `Global config ~/.config/opencode/opencode.json Or project-specific ./opencode.json`
- **Config Format:** TOML-based `opencode.toml`; supports project-level and global overrides.
- **Bring Your Own Keys:** Set `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, or point to Ollama.
- **Launch:** `opencode` in any project directory. Have conda env activated before.

YHK

Plan vs Build Mode

Switch between them using the Tab key.
Plan Mode:

- Read-only - Cannot make changes to your code
- Analyzes and explores your codebase
- Suggests implementation strategies
- Perfect for understanding before modifying

Build Mode (Default)

- Full access - Can read, write, and modify files
- Executes code changes
- Runs commands
- Best for actual development work

Pro Tip: Always start complex features in Plan mode to outline your approach, then switch to Build mode for implementation.

YHK

Configuration

Overview

- OpenCode is fully configurable at **two scopes**: Global (`~/.config/opencode/`) and Project (`.opencode/` in repo root).
- **Config priority** (highest → lowest): CLI flags → Custom env (`OPENCODE_CONFIG_DIR`) → Project `opencode.json` → Global `opencode.json` → Remote org defaults (`.well-known/opencode`).
- **Subdirectories** (plural names preferred): `agents/`, `skills/`, `tools/`, `modes/`, `commands/`, `themes/`, `plugins/`.
- **Schema-validated:** Config is validated against `opencode.ai/config.json` — editors autocomplete fields automatically.
- **Claude Code compatibility:** Falls back to `CLAUDE.md` / `~/.claude/` conventions if OpenCode files are absent.
- Each section below covers: what ships built-in → what you can customize → how to create your own.

YHK

AGENTS.md

AGENTS.md helps OpenCode understand: project structure, Coding conventions, Technology stack etc

```
## Project: My App

## Tech Stack
- Next.js 14 with App Router
:

## Conventions
- Use functional components
:

## Structure
- /app - Next.js app router pages
- /components - Reusable UI components
:
```

Rules — Persistent AI Instructions

- **What:** Plain-text instructions injected into every LLM context. Think of them as standing orders — coding style, conventions, constraints.
- **Built-in bootstrap:** `/init` auto-generates `AGENTS.md` by scanning your repo — languages, deps, folder structure, patterns.
- **Scope:**
  - *Project rules*: `./AGENTS.md` — commit to Git; shared across team.
  - *Global rules*: `~/.config/opencode/AGENTS.md` — personal prefs, never committed.
  - *Claude Code fallbacks*: `CLAUDE.md` / `~/.claude/CLAUDE.md` if no `AGENTS.md` exists.
- **Multi-file rules** via `opencode.json`: glob patterns and remote URLs supported.
- **Lazy loading:** Reference `@rules/style.md` in `AGENTS.md` — agent reads files on-demand, not upfront (saves tokens).
- **Create custom:** Write any Markdown file → list it under `instructions` in `opencode.json`. No special syntax — plain instructions work.

```
{ "instructions": ["docs/standards.md",
    "test/guidelines.md", "packages/*/AGENTS.md",
    "https://myorg.com/team-rules.md"] }
```

Agents — Built-in & Custom

- **Two types:** *Primary agents* (you interact with directly, cycle via `Tab`) and *Subagents* (specialists invoked by primary agents or `@mention`).
- **Built-in Primary Agents:**
  - `Build` — default; all tools enabled; full file + shell access.
  - `Plan` — read-only; all mutating tools set to `ask`; safe for analysis.
- **Built-in Subagents:**
  - `General` — full tool access; multi-step parallel tasks.
  - `Explore` — read-only codebase navigator; fast file/pattern search.
- **Common custom agents:** `Review` (read-only + docs), `Debug` (bash + read), `Docs` (write, no shell), `Security` (audit mode).
- **Create custom:** Run `opencode agents create` → CLI prompts for scope, description, tools, model → writes `.opencode/agents/<name>.md`.
- **Agent frontmatter** fields: `model`, `temperature`, `mode` (`primary`/`subagent`/`all`), `tools`, `permission`, `steps` (max iterations), `color`.
- **Set default agent** in `opencode.json`: `"default_agent": "build"` — applies across TUI, CLI, IDE, GitHub Action.

YHK

Tools — Built-in & Custom

- **What:** Functions the LLM calls to act on your codebase. All built-in tools are enabled by default.
- **Built-in Tools:**
  - `bash` — execute shell commands (`git`, `npm`, `make`, etc.)
  - `edit` / `multiedit` — exact string replacement in files
  - `write` — create/overwrite files
  - `read` — read file contents (supports line ranges)
  - `grep` — regex search across codebase
  - `glob` — find files by pattern (`**/*.ts`)
  - `ls` — list directory contents
  - `patch` — apply unified diff patches
  - `webfetch` — fetch URLs during sessions
  - `skill` — load a `SKILL.md` on-demand
  - `todo` — manage task lists for multi-step work
  - `lsp` — code intelligence (defs, refs, hover) [experimental]
- **Custom Tools:** TypeScript/JS files in `.opencode/tools/` (project) or `~/.config/opencode/tools/` (global). Filename = tool name. Can wrap scripts in any language (Python, Bash, Go, etc.).
- **Override built-ins:** Custom tool with same name replaces the built-in (e.g., `bash.ts` → sandboxed shell).

```
// .opencode/tools/db-query.ts
import { tool } from "@opencode-ai/plugin"
```

YHK

Agent Skills — Reusable Behavior Packages

- **What:** Markdown instruction bundles loaded on-demand by agents via the `skill` tool. Unlike rules (always loaded), skills are lazy — the agent picks them when relevant.
- **No built-in skills ship with OpenCode** — skills are community/team-created. Claude Code-compatible skills from `~/.claude/skills/` work as fallbacks.
- **Discovery locations** (all checked at startup):
  - `.opencode/skills/<name>/SKILL.md` — project-local
  - `~/.config/opencode/skills/<name>/SKILL.md` — global
  - `.claude/skills/` and `.agents/skills/` — compatibility paths
- **Name rules:** Lowercase alphanumeric + single hyphens; 1–64 chars; must match directory name.
- **Permissions:** Control skill access per agent with `allow` / `deny` / `ask` and wildcard patterns (`internal-*`).
- **Create a custom skill:**

```
# .opencode/skills/git-release/SKILL.md
---
name: git-release
description: Create consistent releases and changelogs
license: MIT
compatibility: opencode
```

Permissions — Granular Action Control

- **What:** Per-tool access policy — `allow` (auto-run), `ask` (prompt user), or `deny` (block). Defaults: most tools `allow`; `doom_loop` and `external_directory` default to `ask`; `.env` files denied by default.
- **Scope:** Set globally in `opencode.json`; override per-agent in agent frontmatter. Agent rules take precedence.
- **Runtime `always` approval:** During a session, approve a pattern once → auto-approved for the rest of the session.
- **External directory guard:** Use `external_directory` permission to allow/deny tool calls touching paths outside the working directory.
- **Skill permissions:** Controlled separately — grant/deny specific skills by name or pattern.
- **Wildcard global policy + overrides** (last matching rule wins):

```
{ "permission": {
    "*": "ask",           // ask for everything by default
    "bash": {
      "*": "ask",
      "git *": "allow",   // git commands auto-approved
      "npm *": "allow",
      "rm *": "deny"      // destructive: always blocked
    },
    "edit": {
```

Modes — Contextual Tool Profiles

- **What:** Named configurations combining a system prompt + tool subset + temperature. Switch modes to change agent behavior without switching agents.
- **Difference from Agents:** Agents are full AI configurations with their own identity and model; Modes are lightweight overlays applied on top of an agent.
- **Built-in Modes:**
  - `Build` (default) — all tools on, full write access
  - `Plan` — analysis only; mutating tools set to `ask`
- **Useful custom modes:** `Review` (read-only + docs), `Debug` (bash + read, no writes), `Refactor` (edit only, no shell), `Docs` (write files, no system commands).
- **Create custom mode** — Markdown file in `.opencode/modes/` or `~/.config/opencode/modes/`:
- **Switch modes** in TUI: `Tab` key cycles between `Build` and `Plan` by default.

```
---
name: debug
description: Investigate issues, no file changes
tools:
  bash: true
  read: true
```

Commands — Slash-Invokable Shortcuts

- **What:** Reusable prompt templates invoked with / in the TUI. Eliminate repetitive prompt writing for common workflows.
- **Built-in commands:**
  - `/init` — scan project, generate `AGENTS.md`
  - `/models` — browse and switch active model
  - `/connect` — authenticate a new provider
  - `/share` — generate a shareable session URL
  - `/help` — list all available commands
- **Create custom commands** in `opencode.json` (or as `.opencode/commands/<name>.md` files):
- **`$ARGUMENTS`:** Anything typed after `/command-name` is injected here — makes commands parameterized.
- **Team sharing:** Commit `.opencode/commands/` to Git — entire team gets the same shortcuts.

```
{ "command": {
    "test-report": {
      "description": "Full test suite with coverage",
      "template": "Run the full test suite with
        coverage report and show any failures.
        Suggest fixes for failing tests."
    },
    "new-component": {
```

YHK

MCP Servers — External Tool Integration

- **What:** Model Context Protocol (MCP) servers expose external tools and data sources to OpenCode. Any MCP-compatible server becomes a tool the agent can call natively.
- **Why:** Extends OpenCode beyond the filesystem — databases, APIs, cloud services, monitoring tools, internal systems.
- **Popular MCP servers:** GitHub, GitLab, Slack, PostgreSQL, Puppeteer, Brave Search, Filesystem (extended), AWS, Docker, Linear, Jira, Notion, Sentry.
- **Configure in `opencode.json`:**
- **Tool naming:** MCP tools are prefixed by server name: `github_create_issue`, `postgres_query`, etc. Use `mymcp_*` in permissions to control all tools from one server.
- **Org defaults:** Deploy company-wide MCP servers via `.well-known/opencode` — auto-loaded for all authenticated users.
- **Build your own:** Any HTTP/stdio process implementing the MCP spec becomes a server — SDKs available in TypeScript, Python, Go.

```
{ "mcp": {
    "github": {
      "type": "stdio",
      "command": "npx",
```

YHK

Custom Tools — Extending the Agent's Hands

- **What:** TypeScript/JS files that define new functions the LLM can call. Can wrap any language internally (Python, Bash, Rust, etc.).
- **Placement:** `.opencode/tools/<name>.ts` (project) or `~/.config/opencode/tools/<name>.ts` (global).
- **One file = one or more tools:** Single default export → tool named after file. Multiple named exports → tools named `<file>_<export>`.
- **Multi-tool file example:**
- **Context available:** `context.agent`, `context.session`, `context.directory` (working dir), `context.worktree` (git root).
- **Override built-ins:** Same filename as a built-in (e.g., `bash.ts`) replaces the built-in completely — useful for sandboxing or auditing.
- **Any language via shell:** Call Python/Bash scripts inside `execute()` using `child_process` — tool definition is TS, implementation is anything.
- **Schema validation:** Use `tool.schema` (Zod) for type-safe arg definitions with `.describe()` for LLM-facing documentation.

```
// .opencode/tools/math.ts
import { tool } from "@opencode-ai/plugin"
export const add = tool({ description: "Add two numbers",
  args: { a: tool.schema.number(), b: tool.schema.number() },
  async execute({ a, b }) { return a + b }
```

YHK

Models — Provider & Model Configuration

- **75+ models supported** across: Anthropic, OpenAI, Google Gemini, Groq, Mistral, Cohere, xAI, DeepSeek, Qwen, Ollama (local), Hugging Face, AWS Bedrock, Azure, Vertex AI, and more.
- Chose models with long context window and have access to tools (function calling)
- **Switch models** in TUI: `/models` command → browse and select. Or `-m provider/model` CLI flag at launch.
- **Configure in `opencode.json`:**
- **Variants:** Define multiple configurations of the same model (different temperature, max_tokens, reasoning effort) and cycle with `variant_cycle` keybind.
- **Per-agent model:** Each custom agent can specify its own model — e.g., `Plan` uses a cheap fast model; `Build` uses full Sonnet/Opus.
- **Small model for meta-tasks:** OpenCode uses a lightweight model (default: `gpt-5-nano` via Zen) for session titling and other housekeeping — configurable separately.
- **Authenticate:** `/connect` in TUI → select provider → paste API key. Supports OAuth for GitHub/GitLab providers.

```
1 { "model": "anthropic/claude-sonnet-4-6",
```

YHK

Real-World Use Cases

- **Understanding a Codebase:**
  `You: Explain the authentication flow in this project` OpenCode will analyze your files and provide a detailed explanation of how authentication works.
- **Adding a Feature:** `You: Add a dark mode toggle to the settings page` OpenCode will: Find the settings page, Identify the theme system, Implement the toggle, Update necessary styles
- **Debugging:**
  `You: The login form is not submitting. Here's the error: [paste error]` OpenCode will trace the issue and suggest fixes.
- **Multi-File Refactoring:** Rename, restructure, or migrate patterns across the entire codebase atomically.
- **Documentation Generation:** Auto-generate README, API docs, and inline comments from code context.
- **Test Generation:** Scaffold unit/integration tests with awareness of existing test patterns.
- **Security Auditing:** Identify unsanitized inputs, SQL injection points, or insecure defaults.

YHK

Demo

Initialize & Understand

- **Goal:** Show how OpenCode scans and understands a new project from scratch.
- **Setup:** Open terminal inside any Git repository.
- **Launch:** `$ opencode`
- **Command:** `> /init`
- **What Happens:** OpenCode scans all local files, infers stack (languages, deps, structure), and writes `AGENTS.md`, the AI's instruction manual for this project.
- **Key Point:** Every subsequent interaction is grounded in this context, no hallucinated file paths.

YHK

Fix & Refactor

- **Goal:** Demonstrate AI-driven code modification on a real file.
- **Command:** `> @file:main.go fix the bug where user input isn't sanitized`
- **Agent Action:** Reads `main.go`, identifies the unsanitized input vector, proposes a targeted patch.
- **Review Flow:**
  - Press `Tab` → switch to **Plan** mode: inspect the proposed diff before anything touches disk.
  - Press `Tab` → switch to **Build** mode: apply the change.
- **Key Point:** You stay in control, the agent proposes, you decide.

YHK

Tips for Beginners

- Be Specific: Don't say "Make it better" but say, "Refactor the handleSubmit function to use async/await and add error handling"
- Provide Context: Don't say "Fix the bug" but say, "The login form throws 'undefined is not a function' when clicking submit. The error occurs in auth.ts line 45"
- Use Plan Mode First: For complex features, ask OpenCode to create a plan before implementation.
- Leverage Image Support: Drag and drop screenshots or mockups into the terminal—OpenCode can understand images!
- Create Project-Specific Configs: Different projects may need different settings. Use local opencode.json files.

YHK

Summary

Why Use OpenCode?

- **Privacy-First:** Code never leaves your machine by default; no telemetry, no vendor data retention.
- **Cost Efficiency:** No subscription fees, pay only for actual API token usage; local models are free.
- **No Vendor Lock-in:** Switch AI providers in seconds via config; one tool, all models.
- **Terminal-Native Speed:** Faster context-switch than IDE plugins; integrates directly into your existing shell workflow.
- **Transparent & Auditable:** Fully open-source (Apache 2.0), inspect, fork, or extend anything.
- **Enterprise-Ready:** Supports self-hosted LLMs and air-gapped (physically isolated from all other networks) environments.
- **Bottom Line:** Faster dev loop, zero lock-in, full code privacy, AI that works *with* your terminal.

YHK

Quick Reference — Config Cheat Sheet

- **File locations** (P = project .opencode/, G = global ~/.config/opencode/):
  - Rules: P: AGENTS.md · G: AGENTS.md
  - Agents: P: agents/<name>.md · G: agents/<name>.md
  - Skills: P: skills/<name>/SKILL.md · G: skills/<name>/SKILL.md
  - Tools: P: tools/<name>.ts · G: tools/<name>.ts
  - Modes: P: modes/<name>.md · G: modes/<name>.md
  - Commands: opencode.json → "command": {...}
  - MCP: opencode.json → "mcp": {...}
  - Permissions: opencode.json → "permission": {...}
- **Key CLI commands:**
  - opencode agents create — scaffold a new agent interactively
  - /init — generate AGENTS.md for current project
  - /models — browse and switch models
  - /connect — authenticate a new provider
  - /share — share current session URL
- **Priority order** (highest → lowest): CLI flags → OPENCODE_CONFIG_DIR → .opencode/opencode.json → ~/.config/opencode/opencode.json → .well-known/opencode (org defaults).
- **Docs:** opencode.ai/docs · **GitHub:** github.com/anomalyco/opencode

YHK

Resources

- GitHub: `github.com/opencode-ai/opencode`
- Docs: `opencode.ai/docs`
- NXCode.io "OpenCode Tutorial 2026: Install & Setup After Anthropic Block (Updated Guide)"

YHK